

Hierarchical Coded Computation

Nuwan Ferdinand and Stark C. Draper

Department of Electrical and Computer Engineering, University of Toronto, Toronto, ON, Canada

Email: {nuwan.ferdinand and stark.draper}@utoronto.ca

Abstract—Coded computation is a method to mitigate “stragglers” in distributed computing systems through the use of error correction coding that has lately received significant attention. First used in vector-matrix multiplication, the range of application was later extended to include matrix-matrix multiplication, heterogeneous networks, convolution, and approximate computing. A drawback to previous results is they completely ignore work completed by stragglers. While stragglers are slower compute nodes, in many settings the amount of work completed by stragglers can be non-negligible. Thus, in this work, we propose a hierarchical coded computation method that exploits the work completed by all compute nodes. We partition each node’s computation into layers of sub-computations such that each layer can be treated as (distinct) erasure channel. We then design different erasure codes for each layer so that all layers have the same failure exponent. We propose design guidelines to optimize parameters of such codes. Numerical results show the proposed scheme has an improvement of a factor of 1.5 in the expected finishing time compared to previous work.

I. INTRODUCTION

In cloud-based distributed computing systems slow working nodes, known as stragglers, are a bottleneck that can prevent the realization of faster compute times [1]. Although stragglers cannot be completely eliminated, recent results show that their effect can be minimized through the effective use of error correction codes [2]–[8]. The foundational concept is to introduce redundant computations (additional workers are needed) such that the completion of any fixed-cardinality subset of jobs suffices to realize the desired solution. The idea is easily illustrated through an example [2] of vector-matrix multiplication; the computation of Ax . In this example the distributed system consists of three workers and a master node. The master vertically decomposes the matrix A into two sub-matrices A_1, A_2 so $A = [A_1; A_2]$. It next delegates the following tasks to three workers: the first worker computes A_1x , the second A_2x , and the third $(A_1 + A_2)x$. One can trivially note that outputs of any two completed workers are enough for the master to recover the output. The reader may also observe the use of a (3,2) MDS (maximum distance separable) code. One might further note that the linearity of the vector-matrix computation is important as it dovetails with the linearity of MDS codes.

The example above is from [2], the first work on coded computation which discusses vector-matrix multiplication. In that paper the authors show that latency can be reduced significantly through the use of MDS codes. The ideas were extended to matrix-matrix multiplication based on product codes in [3]. Techniques of vector-matrix multiplication are ex-

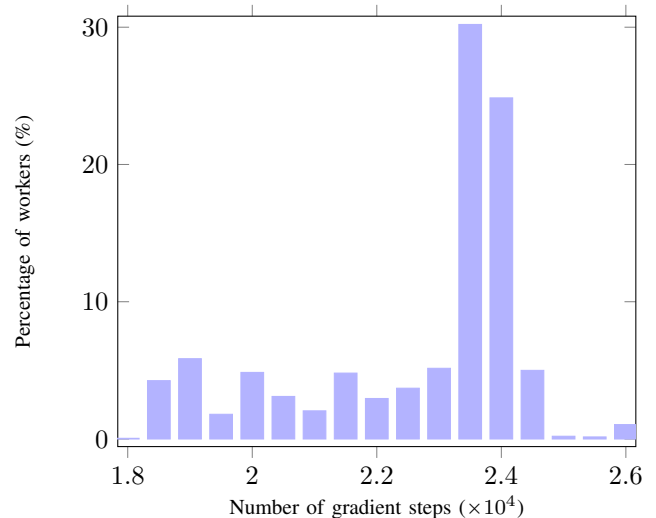


Fig. 1. Histogram: number of completed gradient steps vs percentage of workers. Using Amazon EC2 cloud, 20 machines were given 35 secs to repeatedly compute stochastic gradient steps of a problem of dimension 10^3 . Number of computed gradient steps were counted to find the histogram.

tended in [4] to heterogeneous networks where compute nodes have distinct processing powers. In [5], we proposed anytime coded computation, which significantly reduces the latency through approximate computing, an approach later extended to sequential approximation in [9]. All the above works are based on MDS codes (or product code). They use n workers and the statistic of interest is whether any $k \leq n$ workers finish. Hence, the analysis is based on order statistics. A drawback of all these methods is that they ignore completely work done by the slowest $n - k$ workers. In the case of persistent stragglers—workers that are permanently unavailable—these nodes complete no work. However, in cloud base systems, we rarely experience such persistent stragglers. Rather we observe non-persistent stragglers. Such stragglers are slower, only able to complete partial computation by the time at which the faster workers have completed all their computations. However, in many cloud computing system, the amount of work completed by non-persistent stragglers is non-negligible, thus is wasteful to ignore. We use empirical results from Amazon’s elastic compute cloud (EC2) to illustrate this point. We gave 20 workers 35 secs to compute stochastic gradient steps for a linear regression problem of dimension 10^3 . The histogram of the number of gradient steps computed vs. percentage of workers

is shown in Fig. 1. While the majority of workers were able to finish 23,500 – 26,000 stochastic gradient steps, a significant portion of the workers finished between 18,000 – 23,000. If we classify the latter as non-persistent stragglers, we ignore a significant amount of work. It is the goal of this paper to find a way to exploit that partial work.

In this paper, we propose a hierarchical coding scheme to exploit the work completed by all compute nodes. We do this by exploiting the “sequential” computing nature of each worker. We partition the total computation required of each worker into *layers* of sub-computations. Workers process layers sequentially. Due to this sequential processing, each layer has a different finishing time. I.e., a processor will start to work on the second layer after it finishes the first layer. Therefore, the finishing time of the first layer is lower than that of the second layer. Drawing a parallel with channel coding, the different finishing times of layers create distinct erasure channels. Thus, we encode each layer (or sub-computations) using MDS codes with different rates such that finishing times of all layers are approximately the same. We derive an analytical solution to guide the code design to use at each layer. We show that our method outperforms the earlier approaches.

II. HIERARCHICAL CODED COMPUTATION

Consider a distributed computing system consists of a master and n workers. The goal of the master is to compute a job $g(x)$ where x is the input. We assume that $g(x)$ can be decomposed into k tasks, i.e., $g = \phi(g_1(x), \dots, g_k(x))$. The function $\phi(\cdot)$ maps the set of tasks $\{g_i(x)\}$ to the job $g(x)$. We assume that tasks are linear, i.e., $ag_i(x) + bg_j(x) = (ag_i + bg_j)(x)$. One example is vector-matrix multiplication $g(x) = Ax$. The i -th task here is $g_i(x) = A_i x$ where A_i is the i -th row decomposed sub-matrix of A . In this example $\phi(\cdot)$ simply concatenates the results. Note that in comparison to [2], we decompose the job into a large number of smaller tasks, i.e., $k > n$.

In our approach the master clusters the k tasks into r sets where the j -th set contains k_j tasks. For now, assume that $0 \leq k_j \leq n$. We later detail a procedure to optimize the choice of the k_j . We denote the j -th set by $g^j(x)$. Note that

$$\sum_{j=1}^r k_j = k.$$

We denote the i -th task of the j -th set as $g_i^j(x)$ where $j \in [r]$ and $i \in [k_j]$. Note that we use the notation $[r] = \{1, \dots, r\}$ throughout. The master encodes each set $g^j(x)$ with a length- n MDS code. For the j -th set it uses an (n, k_j) MDS code to generate

$$h^j = \mathcal{E}_j(g^j(x)) \quad (1)$$

where $\mathcal{E}_j$ encodes $g^j(x) = [g_1^j(x), \dots, g_{k_j}^j(x)]$ into $h^j = [h_1^j, \dots, h_n^j]$. We refer h^j as j -th layer. The output length (number of encoded tasks) of each encoded layer is equal to

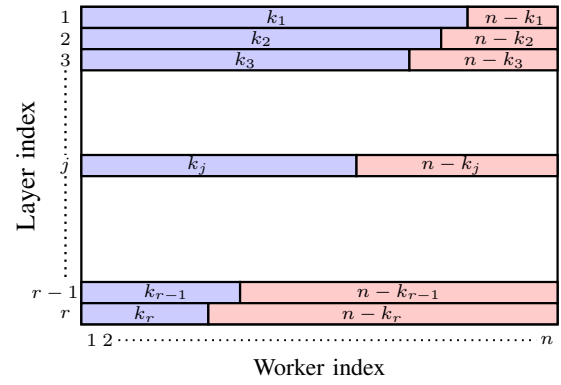


Fig. 2. Tasks allocation to workers. The blue area shows the length of uncoded tasks for different blocks. The red area shows the redundant parity tasks from encoding process. Note that this shows systematic MDS structure, however, it is not necessary.

n . Note that total number of encoded tasks is nr as we have r layers.

Next, the master allocates r encoded tasks to each worker. The i -th worker gets $h_i^1, h_i^2, \dots, h_i^r$. It sequentially computes these tasks and, when each task is complete, transmits the result to the master. That is, the i -th worker first computes h_i^1 , transmits the result to the master. It then computes h_i^2 , transmits the results to master, and so on. The tasks allocation is shown in Fig. 2. Note that we intentionally let $k_{j-1} \geq k_j$ in Fig. 2. The rational for this is as follows. All compute nodes initially work on the first layer $h_i^1, i \in [n]$. They then transmit their results to the master. They next work on h_i^2 , and so forth. Due to the sequential processing nature of the compute nodes, the i -th worker finishes h_i^{j-1} before it finishes h_i^j . Therefore, for any given amount of compute time, each layer has a different probability of finishing. We can conceive of these layers as parallel and independent erasure channels. The top layers are better channels (lower probability of erasure) than the later ones. Thus, we need to allocate less protection for the top layers (we use a higher-rate MDS code) and use more protection (we use a lower-rate MDS code) for the lower layers.

The master sequentially receives the results of the tasks from each worker. To recover the j -th layer and compute $g^j(x)$ it needs to receive at least k_j finished tasks. From any such set it can decode to recover $g^j(x)$ via

$$g_j(x) = \mathcal{D}_j(h_{S_j}^j) \quad (2)$$

where $h_{S_j}^j \subset \{h_1^j, \dots, h_n^j\}$ denotes a subset of any k_j tasks. The decoding function $\mathcal{D}_j$ maps $h_{S_j}^j$ to the $g^j(x)$. Once the master has recovered all the layers, it can obtain the final result $g(x)$.

Remark 1: In this work, each worker sends result of each task to the master before starting to work on the next task. Thus, the outputs of slower workers will also be used. E.g., We want at least k_1 workers to finish the first layer but only need $k_2 \leq k_1$ workers to finish the second layer.

III. FINISHING TIME DISTRIBUTION

In this section, we determine the finishing time distribution of our proposed scheme as a function of $k_1, \dots, k_r$. We then describe a method to optimize the parameters k_j s to maximize the probability of finishing the job for a given time.

A. Finishing time distribution

The job is complete when each of the r layers completes. For layer j to complete at least k_j of the layer- j tasks must complete. In the following we determine the distribution of at least the minimal number of tasks completing for every layer. We call this the “finishing time”. Before deriving the distribution of the finishing time, we make certain assumptions, the same as were made in [2].

Let $F_s(t)$ be the probability that a worker is able to finish s tasks by time t , let

$$F_s(t) = \begin{cases} 1 - e^{-\mu(\frac{t}{s} - \alpha)}, & \text{if } t \geq s\alpha \\ 0 & \text{else} \end{cases}, \quad (3)$$

where μ and α are constants. All workers are assumed to have independent and identical finishing time distributions. Let τ be the finishing time (i.e., at least k_j subtasks finish in every layer $j \in [r]$) at which point the job $g(x)$ can be recovered. The following theorem specifies the distribution of the finishing time.

Theorem 1: Assuming $k_1 \geq k_2, \dots, \geq k_r$, the distribution of τ is

$$\Pr(\tau \leq t) = \sum_{m_1=k_1}^n \sum_{m_2=k_2}^{m_1} \dots \sum_{m_r=k_r}^{m_{r-1}} \prod_{s=0}^r \binom{m_s}{m_{s+1}} (F_s(t) - F_{s+1}(t))^{m_s - m_{s+1}} \quad (4)$$

where $m_0 = n$, $m_{r+1} = 0$, $F_0(t) = 1$, and $F_{r+1}(t) = 0$.

Proof: The detailed proof will be given in the extension of this paper. The proof intuition is as follows. On trivially valid observation is that a worker cannot already have completed s tasks but not $u \leq s$ tasks. Furthermore, we make the following assumptions. Let T_i be a random variable that denotes the completion time of a single task by the i -th worker. As in the previous work, we assume linear scaling of the processing time, i.e., if T_i is the processing time of single tasks, $2T_i$ is the processing time of two equivalent sized tasks. Thus sT_i is the time it takes the i th worker to finish s tasks, then the probability that the i th worker finishes s tasks by time t is equal to $\Pr(T_i \leq t/s) = F_s(t)$.

In order for the master to complete the job, m_1 out of n workers have to finish the first task by time t (where $k_1 \leq m_1 \leq n$). Out of these m_1 workers, m_2 must also complete the second task (where $k_2 \leq m_2 \leq m_1$), and so on. Generally m_j workers must complete the first the j th task where $k_j \leq m_j \leq m_{j-1}$ for all $j \in [r]$. Now we translate this scenario to time distribution. By time t we need $n - m_1$ workers' finishing times to be greater than t , $m_1 - m_2$ workers' finishing times to be between $t/2$ and t , $m_2 - m_3$ workers' finishing times to be

between $t/3$ and $t/2$, and on until $m_{r-1} - m_r$ workers finishing times are between t/r and $t/(r-1)$. The final m_r workers' finishing times must all be less than t/r . This completes the proof sketch.

B. Optimal encoding parameters

Now we find the $k_1, \dots, k_r$ that maximize the probability of finishing the job by time t . This can be formulated as and integer optimization:

$$\begin{aligned} \max_{k_1, \dots, k_r} \quad & \Pr(\tau \leq t) \\ \text{s.t.} \quad & \sum_{j=1}^r k_j = k, \\ & k_j \geq k_i, \quad \forall j \geq i, \\ & k_j \leq n, \quad \forall j \in [r], \\ & k_j \in \mathbb{Z}^+, \quad \forall j \in [r]. \end{aligned} \quad (5)$$

Integer optimization problems are combinatorial in nature and therefore hard to solve for large-scale problems. To solve moderately-sized problems through (slightly smarter) exhaustive search one can impose the following constraint to limit the search space: $k_j \geq k_i, \forall j \leq i$. This constraint is not active due to the fact that initial layers will be finished faster than the later layers (due to the sequential processing nature of the compute nodes) and therefore require less protection. In the next sub-section, we propose an alternative method to find sub-optimal $k_1 \dots k_r$ quickly.

Remark 2: Note that the optimal solution set varies with t .

IV. ASYMPTOTIC ANALYSIS

The alternative method to selecting the $k_1 \dots k_r$ that we outline in this section is first to find the probability that tasks were not complete by time t , i.e., $\Pr(\tau > t)$. We call this the *probability of failure* by time t . We then derive an asymptotic failure probability for large t . We find $k_1, \dots, k_r$ that minimize leading coefficient of asymptotic $\Pr(\tau > t)$. This optimization problem can be formulated as an integer linear program, which can be readily solved. The following theorem describes the asymptotic distribution:

Theorem 2: For large t ,

$$\Pr(\tau > t) = \max_{j \in [r]} \left\{ \binom{n}{k_j - 1} e^{-\frac{\mu(n - k_j + 1)t}{j}} \right\}. \quad (6)$$

Proof: The proof will be given in the extension of this paper.

The failure probability is governed by the smallest coefficient of the failure exponent. We want to choose the $k_1, \dots, k_r$ to minimize (6), which is equivalent to maximizing the smallest coefficient of the failure exponent¹. Before solving this problem, we provide following corollary, which gives the smallest coefficient.

Corollary 3:

$$\lim_{t \rightarrow \infty} \frac{-\log(\Pr(\tau > t))}{t} = \min_{j \in [r]} \left\{ \frac{\mu(n - k_j + 1)}{j} \right\}. \quad (7)$$

¹Note that the constant term is negligible when $t \rightarrow \infty$

We are now ready to state the optimization problem:

$$\begin{aligned}
 & \max_{k_1, \dots, k_r} \quad \min_{j \in [r]} \left\{ \frac{(n - k_j + 1)}{j} \right\} \\
 & \text{s.t.} \quad \sum_{j=1}^r k_j = k, \\
 & \quad k_j \geq k_i, \quad \forall j \geq i, \\
 & \quad k_j \leq n, \quad \forall j \in [r], \\
 & \quad k_j \in \mathbb{Z}^+, \quad \forall j \in [r]
 \end{aligned} \tag{8}$$

We can transform above optimization problem to a linear program as

$$\begin{aligned}
 & \max \quad z \\
 & \text{s.t.} \quad \sum_{j=1}^r k_j = k, \\
 & \quad z \leq \frac{(n - k_j + 1)}{j}, \quad \forall j \in [r], \\
 & \quad k_j \geq k_i, \quad \forall j \geq i, \\
 & \quad k_j \leq n, \quad \forall j \in [r], \\
 & \quad k_j \in \mathbb{Z}^+, \quad \forall j \in [r]
 \end{aligned} \tag{9}$$

This is a linear program with integer constraints on the k_j . By relaxing the integer constraint we get a linear program.

Robustifying to persistent stragglers: The finishing time distribution of practical cloud computing systems may have a long tail due to persistent stragglers. The shifted exponential model we considered in above does not reflect this behavior. Thus, $k_j = n$ is a possible solution to (9). To robustify the solution to the possible presence of persistent stragglers we change the optimization problem in (9) to

$$\begin{aligned}
 & \max \quad z \\
 & \text{s.t.} \quad \sum_{j=1}^r k_j = k, \\
 & \quad z \leq \frac{(n - k_j + 1)}{j}, \quad \forall j \in [r], \\
 & \quad k_j \geq k_i, \quad \forall j \geq i, \\
 & \quad k_j \leq n - S, \quad \forall j \in [r], \\
 & \quad k_j \in \mathbb{Z}^+, \quad \forall j \in [r].
 \end{aligned} \tag{10}$$

This yields a solution that is robust up to S stragglers.

V. EVALUATION

In this section, through application of (9), we evaluate the probability of failure, expected finishing time, and the leading coefficient of the failure exponent. We compare our result to those of [2] and to uncoded computation. For a fair comparison, we fix the number of workers in all schemes and each worker is given same computation load. If we assume that the computation load of the job $g(x)$ is $\mathcal{O}(\gamma)$, then in our scheme, each task has a computation load of $\mathcal{O}(\gamma/k)$ as the job is divided into k tasks. As each worker gets r tasks, the computation load of each worker is $\mathcal{O}(\gamma r/k)$. We can get the

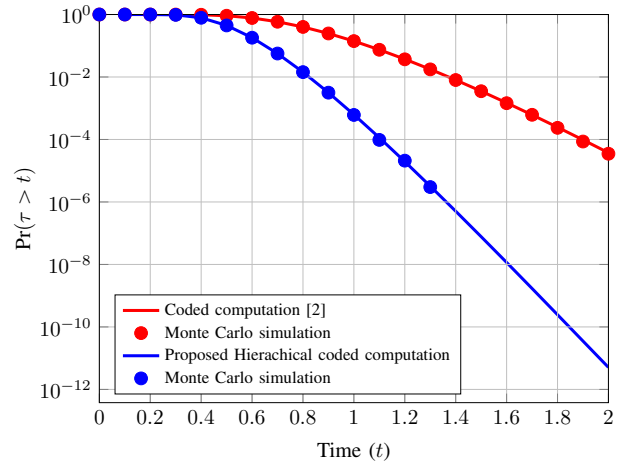


Fig. 3. Failure probability vs time. We used the following values: $n = 20$, $k = 100$, $\mu = 0.1$, and $\alpha = 0.01$, $r = 10$, $k_1 = 19$, $k_2 = 17$, $k_3 = 15$, $k_4 = 13$, $k_5 = 11$, $k_6 = 9$, $k_7 = 7$, $k_8 = 5$, $k_9 = 3$, and $k_{10} = 1$.

same computation load in [2] by dividing the job $g(x)$ into k/r tasks such that computation load of each worker is $\mathcal{O}(\gamma r/k)$. Thus, we use $(n, k/r)$ MDS code for [2] in simulations.

A. Probability of failure and expected finishing time

We fixed the number of workers to be $n = 20$. We used (9) to find the k_j s for various r and picked the r that maximizes z in (9). Fig. 3 plots the failure probability vs time. The solution set to (9) is provided in the caption. At a failure probability of 10^{-4} , we observe 0.8 secs speed up compared to [2]. This is equivalent to a 42% improvement. We included Monte Carlo simulations to corroborate the analytical results.

Fig. 4 illustrates the expected finishing time vs the number of tasks. For all values of k , our scheme has a 1.5 factor improvement in expected time. Note that the selection of the solution set $k_1, \dots, k_r$ is based on the failure exponent. Thus, it is not necessary the solution set that minimize the expected time. We expect further improvement in finishing time if we were to optimize to minimize the expected finishing time.

B. Failure exponent comparison

In this section we compare the leading coefficients of the failure probability exponents. Let $k_1^* \dots k_r^*$ be the solution to (9). Then, the leading coefficient L of our hierarchical coded computation is

$$L = \min_{j \in [r]} \left\{ \frac{\mu(n - k_j^* + 1)}{j} \right\}. \tag{11}$$

As discussed at the beginning of this section, we used as $(n, k/r)$ MDS code for [2] to get a fair comparison. Let τ_p be the finishing time of the $(n, k/r)$ coded computation scheme from [2]. Then, the leading coefficient L_p of failure exponent [2] is given by

$$L_p = \lim_{t \rightarrow \infty} \frac{-\log(\Pr(\tau_p > t))}{t} = \frac{\mu(n - k/r + 1)}{r}. \tag{12}$$

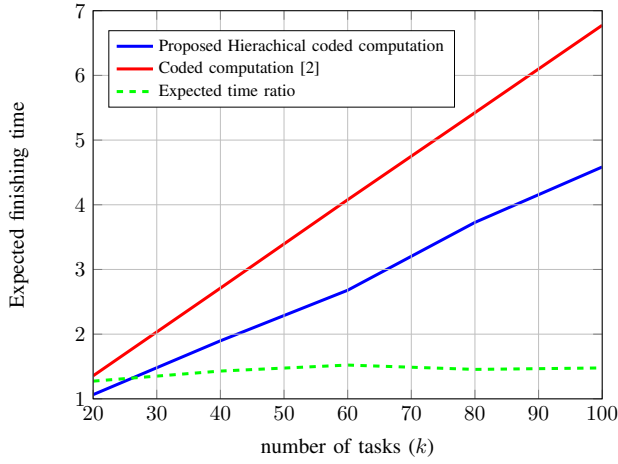


Fig. 4. Expected time vs. number of tasks. We used the following values: $n = 20$, $\mu = 0.1$, and $\alpha = 0.01$. For different k , the r that maximizes z in (9) is used.

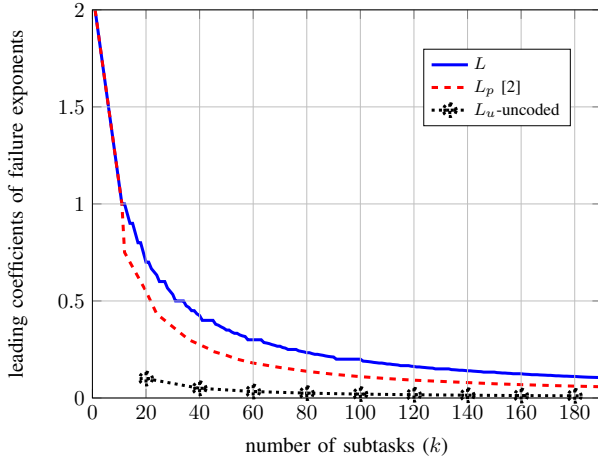


Fig. 5. Leading coefficients of the failure exponents comparison. We used the following values: $n = 20$, $\mu = 0.1$. We used the r that maximizes z in (5) for respective k .

In Fig. 5 we compare L and L_p for different values of k . When $k \leq n/2 = 10$ both schemes have same leading coefficients. This is expected as when there are a small number of subtasks, there is no flexibility to exploit by coding across layers. However, our proposed hierarchical coded computation outperforms [2] for $k \geq n/2$. We also plot the leading coefficient of the uncoded scheme, which is $L_u = \mu n/k$ for $k/n \in \mathbb{Z}$. In order to quantify the gain, we plot the ratio L/L_p in Fig. 6. We observe a 1.8 improvement factor in hierarchical coded computation, when compared to coded computation [2].

C. Complexity

In [2], the decoding complexity is mainly contributed by inverting a $k/r \times k/r$ matrix. In our case, we have to decode r independent MDS codes, which can be done in parallel. As $k_1 \geq k_j$, $j \in \{2, \dots, r\}$ by design, the complexity of our method is governed by inverting a $k_1 \times k_1$ matrix. As $k_1 \geq$

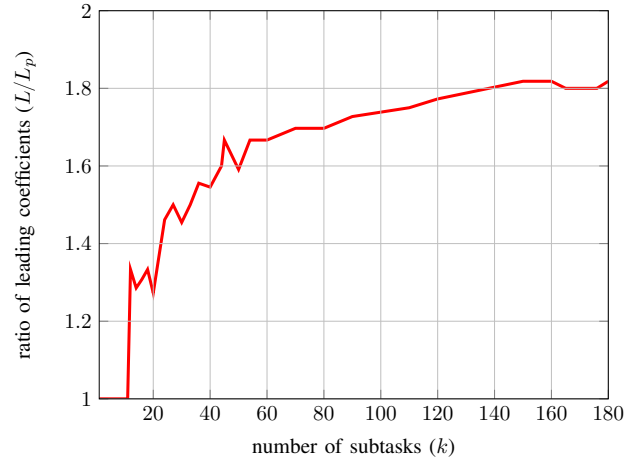


Fig. 6. Ratio of leading coefficients of the failure exponents (L/L_p).

k/r , we have slightly higher complexity when compared to [2]. However, note that the complexity remains lower than the number of workers as $k_1 \leq n$.

VI. CONCLUSION AND FUTURE EXTENSIONS

Our proposed hierarchical coded computation scheme can be used in any situation where coded computation [2] can be used, and at a lower latency. Numerical results show a 1.5 factor improvement in the expected computation latency. Furthermore, the hierarchical coded computation provides additional benefits in a range of other applications including non linear functions with linear components, sequentially ordered tasks where the master needs to output tasks sequentially, and approximate computing where some tasks have greater impact.

REFERENCES

- [1] J. Dean, G. S. Corrado, R. Monga, K. Chen, M. Devin, Q. V. Le, M. Z. Mao, M. Ranzato, A. Senior, P. Tucker, K. Yang, and A. Y. Ng, "Large scale distributed deep networks," in *Proc. of the Int. Conf. Neural Inf. Proc. Sys.*, 2012, pp. 1223–1231.
- [2] K. Lee, M. Lam, R. Pedarsani, D. Papailiopoulos, and K. Ramchandran, "Speeding up distributed machine learning using codes," in *IEEE Int. Symp. Inf. Theory (ISIT)*, July 2016, pp. 1143–1147.
- [3] K. Lee, C. Suh, and K. Ramchandran, "High-dimensional coded matrix multiplication," in *IEEE Int. Symp. Inf. Theory (ISIT)*, June 2017, pp. 2418–2422.
- [4] A. Reisizadeh, S. Prakash, R. Pedarsani, and S. Avestimehr, "Coded computation over heterogeneous clusters," in *IEEE Int. Symp. Inf. Theory (ISIT)*, June 2017, pp. 2408–2412.
- [5] N. S. Ferdinand and S. C. Draper, "Anytime coding for distributed computation," in *Allerton Conf. on Commun., Control, and Comp.*, Sept 2016, pp. 954–960.
- [6] A. Severinson, A. G. i Amat, and E. Rosnes, "Block-diagonal coding for distributed computing with straggling servers," *ArXiv*, vol. abs/1701.06631, 2017.
- [7] S. Li, M. A. Maddah-Ali, and A. S. Avestimehr, "Fundamental tradeoff between computation and communication in distributed computing," in *2016 IEEE Int. Symp. Inf. Theory (ISIT)*, July 2016, pp. 1814–1818.
- [8] S. Dutta, V. Cadambe, and P. Grover, "Coded convolution for parallel and distributed computing within a deadline," in *IEEE Int. Symp. Inf. Theory (ISIT)*, June 2017, pp. 2403–2407.
- [9] J. Zhu, Y. Pu, V. Gupta, C. Tomlin, and K. Ramchandran, "A Sequential Approximation Framework for Coded Distributed Optimization," *ArXiv e-prints*, Oct. 2017.